PHP AND WORDPRESS DEVELOPMENT · BEFORE YOU BEGIN

Set Up Your Local WordPress for the Course

About 45 minutes to do **before the first session**, most of it downloading: install Docker, unpack the course's Compose project `pagesco-dev`, start it, install WordPress 7.1 on PHP 8.5 with two commands, and run a short check that tells you (and your instructor) whether your laptop is ready. These are the same steps as Lesson 1.2 on the course site (rays-php-wordpress.netlify.app, lesson "Your Local Development Environment"), which shows every step with its real output. Session 1 then spends its time on WordPress instead of progress bars. A second, shorter setup (Node 24, Composer and, if you join for Part 2, a ready-made starting site) comes before Part 2: see **Part 2 Setup** near the end.

⏱️ **30–60 min**, mostly downloading · About 3 GB of images · Windows (WSL 2) · Mac · Linux · Free software

> **PLEASE DO THIS AT HOME, NOT IN CLASS**
>
> The Session 1 images come to about 3 GB on disk. Twenty laptops pulling them at once over venue Wi-Fi takes most of a session. If a step fails, keep going with the others and bring the **exact error message** (a screenshot is fine) to Session 1: its first half hour includes the environment check, and Lesson 1.2's time in the room is an install clinic.

WHAT YOU NEED

- **A laptop** and its **charger**: Windows 10/11 (64-bit) with WSL 2, a recent macOS (Apple silicon or Intel), or a 64-bit Linux. **8 GB of memory or more** is comfortable (the course site itself runs in 2 to 3 GB) and about **10 GB of free disk space** for Docker, the images and the project. You need permission to install software (Docker needs administrator rights once). Chromebooks, tablets and locked work laptops cannot run the course comfortably: talk to your instructor first.
- **Docker**: Docker Desktop on Windows and macOS, Docker Engine with the Compose plugin on Linux (Step 1). Docker Desktop is free for personal use, education and small businesses; larger companies need a paid subscription, so check before you install it on a work computer.
- **A terminal**: the **Ubuntu** app on Windows (WSL 2), Terminal on macOS, your usual terminal on Linux. Every course command runs there.

- **A code editor**: **VS Code** (free) works well, with a PHP extension such as PHP Intelephense; on Windows also the **WSL** extension.
- **A web browser** with developer tools (any current Chrome, Edge, Firefox or Safari).
- **The course's `labs/` folder**, which your instructor sends with this guide. Its file `labs/module-01/l1_0-orientation/check_environment.sh` is the environment check in Step 5.
- **A learning journal**: a plain text file or a paper notebook. Every lesson ends with a journal prompt.

AM I READY FOR THIS COURSE?

PHP and WordPress Development builds on **PHP Foundations** and **MySQL Foundations** (or the same material learned elsewhere): it does not re-teach PHP. Lesson 1.3 is a short refresher on how WordPress uses it. You are ready if you can:

- ☐ Write a PHP function with typed parameters, loop over an associative array and explain what `htmlspecialchars()` protects against
- ☐ Read a small PHP class with a namespace, and say what Composer's `vendor/autoload.php` does
- ☐ Write a `SELECT` with a `JOIN` and a `WHERE`, and say why prepared statements matter
- ☐ Write a small HTML page with a stylesheet, and read basic JavaScript (Module 6 builds blocks with it)
- ☐ Move between folders in a terminal with `cd`, list files with `ls` and run a command someone gives you

Two or more unchecked? Come anyway and tell your instructor. Session 1 includes a short, ungraded diagnostic on these same skills, so your instructor knows where to help.

SET UP IN 5 STEPS

1 Install Docker

~15 min

Skip this step if `docker compose version` already prints a version 2 or newer (the Docker lessons of PHP Foundations or MySQL Foundations may have set you up).

- **Windows 10 and 11: WSL 2 plus Docker Desktop.** Open PowerShell as administrator and run `wsl --install`; restart when asked and create a Linux user name and password when Ubuntu starts for the first time. Install **Docker Desktop for Windows** with the WSL 2 backend (the installer's default), start it, open Settings > Resources > WSL integration and switch on your Ubuntu distribution. From now on, open the **Ubuntu** app for every command, and keep the project in the **Linux file system** (your Linux home folder, `~`), not under `/mnt/c/`: there every page load crosses from Linux to Windows and WordPress becomes many times slower. Inside Ubuntu you are on Linux: do the Linux part of Step 2 too.
- **macOS.** Install **Docker Desktop for Mac** (Apple silicon or Intel, to match your Mac), start it once from Applications and wait until it says it is running. All the course images run natively on both kinds of Mac.
- **Linux.** Install Docker Engine and the Compose plugin from Docker's own package repository (docs.docker.com/engine/install; your distribution's packages are often too old). Then let your user run Docker without `sudo`, and log out and back in:

  ```
  sudo usermod -aG docker "$USER"
  ```

Check (your version numbers will differ):

```
docker --version
docker compose version
docker run --rm hello-world
```

Write `docker compose` with a space. The old separate program `docker-compose`, with a hyphen, is retired.

2 Get the course project (and, on Linux or WSL, run WordPress as you)

~5 min

The course ships a ready-made Compose project, `pagesco-dev`: WordPress 7.1 on PHP 8.5 with Apache, MySQL 9.7, WP-CLI and Mailpit, the same images the lessons were tested on. Download and unpack it in your home folder (on Ubuntu and WSL, `sudo apt install unzip` first if `unzip` is missing):

```
cd ~
curl -LO https://rays-php-wordpress.netlify.app/downloads/pagesco-dev.zip
unzip pagesco-dev.zip
cd pagesco-dev
```

From here on, **your project folder** means `pagesco-dev`, and every `docker compose` command runs inside it.

> LINUX AND WINDOWS (WSL 2) ONLY: BEFORE THE FIRST START
>
> Without this, the WordPress image creates every file as its web server user (uid 33) and your editor gets "Permission denied" in `wordpress/wp-content/`. From the project folder, once:
>
> ```
> printf 'PAGESCO_UID=%s\nPAGESCO_GID=%s\n' "$(id -u)" "$(id -g)" > .env
> cp config/compose.linux.yaml compose.override.yaml
> mkdir wordpress
> cat .env
> ```
>
> `cat .env` prints your two numbers (often `PAGESCO_UID=1000` and `PAGESCO_GID=1000`). Already started without it? The repair, from the project folder:
>
> ```
> docker compose run --rm --user root wpcli chown -R "$(id -u):$(id -g)" /var/ww
> printf 'PAGESCO_UID=%s\nPAGESCO_GID=%s\n' "$(id -u)" "$(id -g)" > .env
> cp config/compose.linux.yaml compose.override.yaml
> docker compose up -d
> ```
>
> macOS (and Docker Desktop without WSL) needs nothing here.

3 Start the site and install WordPress ~5 min (the first start downloads the images)

```
docker compose up -d
docker compose ps
```

`ps` lists `pagesco-db-1` (healthy), `pagesco-mailpit-1` and `pagesco-wordpress-1`. Then install WordPress and turn on pretty permalinks (two long lines; copy them whole):

```
docker compose run --rm wpcli wp core install --url=http://localhost:8080 --
title="Pages & Co." --admin_user=admin --admin_password=password --
admin_email=wordpress@example.com --skip-email
docker compose run --rm wpcli wp rewrite structure '/%postname%/'
```

Expected: `Success: WordPress installed successfully.`, `Success: Rewrite structure set.` and `Success: Rewrite rules flushed.` (The first `run wpcli` builds the course's WP-CLI image, about a minute.) Open **http://localhost:8080**: the Pages & Co. site in Twenty Twenty-Five. Log in at **http://localhost:8080/wp-admin/** as **admin** / **password** (a development password for a site that exists on your computer alone).

Make the course's **wp shortcut** and add the same line to `~/.bashrc` (macOS: `~/.zshrc`) so a new terminal has it too:

```
alias wp='docker compose --progress quiet run --rm wpcli wp'
```

Check: `wp core version --extra` starts with `WordPress version: 7.1.3`, and `wp eval 'echo "PHP ", PHP_VERSION, "\n";'` prints `PHP 8.5.12`.

4 Turn on email (Mailpit) and put `labs/` in place

~3 min

Mailpit catches every email the site sends (nothing is delivered) and shows it at **http://localhost:8025**. WordPress needs one small must-use plugin to reach it; without it `wp_mail()` returns false (`Invalid address: (From): wordpress@localhost`). From the project folder:

```
mkdir -p wordpress/wp-content/mu-plugins
cp config/mailpit-smtp.php wordpress/wp-content/mu-plugins/
wp plugin list --status=must-use
```

The list shows `mailpit-smtp` as `must-use`. Now copy the `labs/` **folder** from your instructor into the project folder, next to `compose.yaml` (not inside `wordpress/`, which is the web root):

```
pagesco-dev/
  compose.yaml   config/   wordpress/   labs/     <- here
```

5 Run the environment check

~2 min

On your computer (not in a container), from the project folder, with the site started:

```
cd ~/pagesco-dev
bash labs/module-01/l1_0-orientation/check_environment.sh
```

It checks Docker and Compose, the project folder, the running services, who owns `wordpress/`, WP-CLI, WordPress 7.1, PHP 8.5 on the web server and in WP-CLI, the site address, permalinks, the home page, `wp cron test`, the debug settings, a test email through Mailpit and the host ports. It changes nothing on your site. Each line says **OK**, **CHECK** (works, read the note), **INFO** or **FIX** (what to do next); the last line is the verdict.

> ✅ DONE WHEN THE LAST LINE SAYS READY FOR PHP AND WORDPRESS DEVELOPMENT
>
> Send that line to your instructor. **READY WITH NOTES** is fine: read the CHECK lines (for example PHP 8.4 instead of 8.5, or a project folder on the Windows drive). **NOT READY**? Fix the **FIX** lines from the top (each names the next step; the table below has more), run the check again, then send the whole output.

The course is built and checked on **WordPress 7.1.3 and PHP 8.5**; keep the image names in `compose.yaml` as they are. A site on **PHP 8.4** runs the course: the lessons note where its output differs (a few cards of Lessons 1.2 and 1.3 show PHP 8.5-only output, such as the longer fatal-error format), and the check says **READY WITH NOTES**. Local, XAMPP, MAMP, Laravel Herd, `wp-env` and WordPress Playground all run WordPress, and Lesson 1.2 shows wp-env and Playground; but the course's outputs, paths (`/var/www/html/...`), port, mail catcher and WP-CLI fixes are tested on the Docker project. If you are new to all of them, use the course project.

COMMON INSTALL PROBLEMS

YOU SEE…	TRY THIS
`Bind for 0.0.0.0:8080 failed: port is already allocated` (or 8025)	Something else listens on that port: another Compose project, an old container, a local Apache, another course's server. Find it with `docker ps` (the PORTS column) and stop it. Changing the course port means changing the site URL too, and the lessons' URLs no longer match: freeing the port is easier.
Linux/WSL: `mkdir: cannot create directory ‘wordpress/wp-content/mu-plugins’: Permission denied`	The site started without the Linux setup: the repair box in Step 2.
`required variable PAGESCO_UID is missing a value`	`compose.override.yaml` is there but `.env` is not: the `printf` line of Step 2.
WSL: everything works but pages take seconds; the check says `CHECK the project is on the Windows drive`	The project is under `/mnt/c/`. Move it to your Linux home folder (`~/pagesco-dev`) and open it with `code .` from Ubuntu.
`no configuration file provided: not found`	The terminal is not in the project folder: `cd ~/pagesco-dev`.
`wp_mail()` returns `bool(false)`; the check says `FIX wordpress/wp-content/mu-plugins/mailpit-smtp.php is missing`	Step 4's `cp`. WordPress's default From address `wordpress@localhost` is rejected; the mu-plugin sends through Mailpit with a valid address.
`Access denied for user 'wordpress'` after editing `compose.yaml`	Database passwords are set on the very first start only. Put the old value back, or start over with `docker compose down -v` (this deletes the database) and Step 3 again.

YOU SEE…	TRY THIS
`Deprecated: Using null as an array offset … php-cli-tools/lib/cli/Colors.php` after every WP-CLI line	You ran a WP-CLI installed on your computer with PHP 8.5: that notice is WP-CLI's own code, not yours. Use the course's `wp` shortcut (the container's WP-CLI hides it).
`npx wp-env` prints something odd ("Security PoC")	That is a different npm package. The WordPress tool is always `npx @wordpress/env …` (Lesson 1.2 and Lesson 8.1).
Docker is installed but `docker ps` fails	Start Docker Desktop. On Linux: `sudo usermod -aG docker "$USER"`, then log out and in.

More cases, each with the exact error text: Lesson 1.2's Common Mistakes section, and the course's **Troubleshooting** page (rays-php-wordpress.netlify.app/troubleshooting.html) and **Common Errors** page (rays-php-wordpress.netlify.app/common-errors.html).

IMAGES AND DOWNLOADS BY SESSION

Docker pulls each image once; after that the course runs offline except where this table says otherwise.

IMAGE	FIRST NEEDED	WHAT FOR
`wordpress:7.1-php8.5-apache`	Session 1	The site: Apache, PHP 8.5.12, WordPress 7.1.3
`mysql:9.7`	Session 1	The database
`wordpress:cli-php8.5`	Session 1	The base of the course's WP-CLI image `pagesco-wpcli:php8.5`, built on your computer by the first `docker compose run wpcli`
`axllent/mailpit:latest`	Session 1	The mail catcher (http://localhost:8025)
`composer:2`	Session 2 (Lesson 1.3), then Modules 5 to 8	Composer: the plugin's autoloader (Lesson 5.1), coding standards and tests (Lesson 8.1)
`node:24` (or Node.js 24 on your computer)	Session 11 (Lesson 6.1), Modules 6 to 8	Building blocks with `@wordpress/scripts`; `npx @wordpress/env` in Lesson 8.1

Internet needed in these lessons: Lesson 1.3 installs the Spanish language pack (`es_ES`) and runs Composer; Lesson 5.1 runs Composer and Lessons 8.1 and 8.4 download Composer packages; Module 6 and Lesson 8.1 download npm packages; Lesson 8.1's wp-env downloads its own images. The supplier API of Lesson 7.3 is a sandbox on your own site (no internet).

PART 2 SETUP (BEFORE SESSION 9)

Part 2, "Plugins to Launch" (Sessions 9 to 18), uses the same project and adds **Composer** (from Session 9), **Node 24** (from Session 11) and a site that has **finished Part 1**. Session 9 opens with the Part 2 check. Joining for Part 2 only? Do Steps 1 to 5 above first, then B below.

A Composer and Node 24

~10 min, mostly downloading

The course runs both in containers, so nothing needs installing on your computer:

```
docker pull composer:2
docker pull node:24
```

Node.js 24 LTS on your computer (for example `nvm install 24`) also works and is a little faster; without it, the lessons' `docker run … node:24 npm …` commands do the same job (Lesson 6.1). Together the two images take about 2 GB on disk: pull them at home.

B The Part 2 starting site (only if your site has not finished Lesson 4.3)

~1 min

Part 2 starts from the site exactly as **Lesson 4.3** leaves it: the `pagesco` block theme active, `pagesco-classic` and `pagesco-tt5-child` installed, the store notice `mu-plugins/pagesco-site.php`, the users maria (editor) and devon (author), the pages About Us, Contact and Events, and the News posts. Learners who did Part 1 already have it. Anyone new, or with a broken Part 1 site, installs the course download **pagesco-part2-start.zip**:

1. Download and unzip it (or use the copy your instructor gives you):

```
curl -LO https://rays-php-wordpress.netlify.app/downloads/pagesco-part2-start.
unzip pagesco-part2-start.zip
```

2. Copy the `pagesco-part2-start` folder into your project's `wordpress/` folder:

```
cp -R pagesco-part2-start ~/pagesco-dev/wordpress/
```

3. From the project folder, run the installer in the WP-CLI container:

```
cd ~/pagesco-dev
docker compose run --rm wpcli sh pagesco-part2-start/install.sh
```

   It saves your current database as `wordpress/pagesco-part2-start/backup-before-part2.sql`, copies the themes and must-use plugins into `wp-content` (replacing those three theme folders), imports `database.sql` and keeps your site address (it runs `wp search-replace` when yours is not `http://localhost:8080`).

4. Log in at **http://localhost:8080/wp-admin/** as **admin** / **password** (**maria** / **maria-pass** and **devon** / **devon-pass** are the course's editor and author).

The import **replaces** your posts, pages, users and settings. Undo: `docker compose run --rm wpcli wp db import pagesco-part2-start/backup-before-part2.sql`. The Linux/WSL setup of Step 2 must be in place, or copying into `wordpress/` fails with "Permission denied". The download's `README.txt` has the same steps.

C The Part 2 check

~2 min

```
cd ~/pagesco-dev
bash labs/module-01/l1_0-orientation/check_environment.sh --part2
```

The Session 1 checks plus three lines at the end: Node 24, the `composer:2` image, and the site. Typical, on a laptop without Node installed:

```
CHECK no Node 24 on this computer (v20.20.2): use the node:24 image as Lesson
6.1 shows (docker run ... node:24 npm ...)
OK    Composer version 2.10.3 (the composer:2 image, Lesson 5.1)
OK    the site as Lesson 4.3 left it: pagesco active, pagesco-classic
installed, mu-plugins/pagesco-site.php
READY WITH NOTES for PHP and WordPress Development
```

> ✅ READY FOR PART 2
>
> Send the last line to your instructor: **READY** or **READY WITH NOTES**. A **FIX** on the site line ("the site is not where Part 2 starts") means B above; a FIX on Composer means `docker pull composer:2`.

THE COURSE AT A GLANCE

SESSIONS	YOU'LL LEARN
1–3	**Part 1: Themes.** WordPress for developers and your local environment, PHP the WordPress way, the content model, core architecture, the database and hooks
4–5	Theme anatomy and a block theme of your own (Checkpoint 1), patterns, style variations and child themes
6–8	The Loop and custom queries in a classic theme, functions.php, then the Part 1 project and assessment
9–11	**Part 2: Plugins to Launch.** Your first plugin, custom post types, security, the Settings API, custom tables, shortcodes, block setup
12–14	Static, dynamic and interactive blocks (Checkpoint 2), the REST API, custom endpoints, external APIs and cron
15–16	Debugging, testing, performance, distribution and deployment

SESSIONS	YOU'LL LEARN
17–18	Your final project, Reserve & Pick Up, then the showcase and next steps

18 sessions of 3 hours in two parts (some programs run Part 1, Sessions 1 to 8, on its own). Every lesson builds the same fictional bookstore website, Pages & Co. Between sessions you finish each lesson's project, exercises, quiz and journal entry on the course site: about two to three hours a week.

FREQUENTLY ASKED

DOCKER DESKTOP USES A LOT OF MEMORY. CAN I LIMIT IT?

Yes. On Windows, WSL 2 takes what it needs up to a limit you set in a `.wslconfig` file in your Windows user folder (`memory=4GB` under `[wsl2]`); on macOS, Docker Desktop's Settings > Resources has a memory slider. Run `docker compose stop` when you are not working.

DOES STOPPING THE SITE LOSE MY WORK?

No. `docker compose stop` and `docker compose down` keep your files (in `wordpress/`) and your database (in a Docker volume); `docker compose up -d` brings it all back. Only `docker compose down -v` deletes the database.

WHERE IS PHPMYADMIN?

Not included: `wp db query` and `wp db cli` cover what the lessons need, with the SQL you know from MySQL Foundations.

IS THE COURSE DATA REAL?

No. Pages & Co., its staff, events and customers, the Part 1 project and assessment clients and the final project's shop are invented for the course (the books are real classics). The passwords are development passwords for a site on your own computer: never use them anywhere else.

WHAT IF A STEP FAILS AND I CANNOT FIX IT?

Come anyway, with the output of the environment check. Session 1 opens with the check and has an install clinic, and you can pair with a neighbor until your machine is ready.

FOR THE FACILITATOR: ROOM SETUP

This section is for the instructor; it can stay on the learners' copy or be left off when you print it.

- ☐ **A week before Session 1:** send this guide (or the two-page Quick Setup with it) and the `labs/` folder without the `*_solution.*` files and `*_solution/` folders. Ask each learner to put `labs/` in `~/pagesco-dev`, run `bash labs/module-01/l1_0-orientation/check_environment.sh` and reply with its last line. Keep a list: no reply, or NOT READY, means an install risk to chase before the day.
- ☐ **Images offline (the fallback when venue Wi-Fi fails):** on a machine that has run the course, save the images to one file and copy it to USB sticks or a shared folder:

```
docker save -o pagesco-images.tar wordpress:7.1-php8.5-apache mysql:9.7
wordpress:cli-php8.5 axllent/mailpit:latest composer:2 node:24
```

Learners load it with `docker load -i pagesco-images.tar` (about 5 GB with all six images; the first `docker compose run wpcli` then builds `pagesco-wpcli:php8.5` from `wordpress:cli-php8.5` without a download). Put `pagesco-dev.zip`, `pagesco-part2-start.zip`, the `labs/` folder (as above, without the checkpoint and assessment folders until they are used) and the workbook PDF on the same stick.

- ☐ **Your own machine:** the same `~/pagesco-dev` project, the check ends with READY, so you can show what it looks like. From the kit folder, `WP_LABS_COMPOSE=~/pagesco-dev python3 labs/check_labs.py module-01` runs every Session 1 lab on your site and puts the database back afterwards (see the kit README); run each session's check before that session.
- ☐ **Projector:** VS Code on `pagesco-dev` at a large font, two terminals in the project folder at 18 pt or more (one for `wp` commands, one for `tail -f wordpress/wp-content/debug.log`), a browser on http://localhost:8080 and Mailpit, the session's deck from `slides/` and the lesson pages open.
- ☐ **The room:** a power socket for every laptop (bring power strips; Docker drains batteries), a whiteboard, sticky notes (help flags and exit tickets), printed workbooks or a shared PDF, and Wi-Fi for the lesson pages. Sessions 2, 11, 12, 15 and 16 download language packs, Composer or npm packages: test the venue network beforehand, and have a phone hotspot as the fallback.
- ☐ **Install trouble on the day:** do not debug one machine in front of the room. Two minutes per machine during the opening check, then pair the learner with a ready neighbor; Lesson 1.2's install clinic is for the rest. The Linux file-owner repair and the port fix take a minute; an image download on venue Wi-Fi does not (use the USB stick). The Session 1 facilitator guide has the full trap table.
- ☐ **Before Part 2:** send the Part 2 Setup section with Session 8's close (or a week before Session 9 to newcomers, with `pagesco-part2-start.zip`). Session 9 opens with `check_environment.sh --part2`.